

Ex. No 10: Malware Disassembly and Analysis using Ghidra

Date:

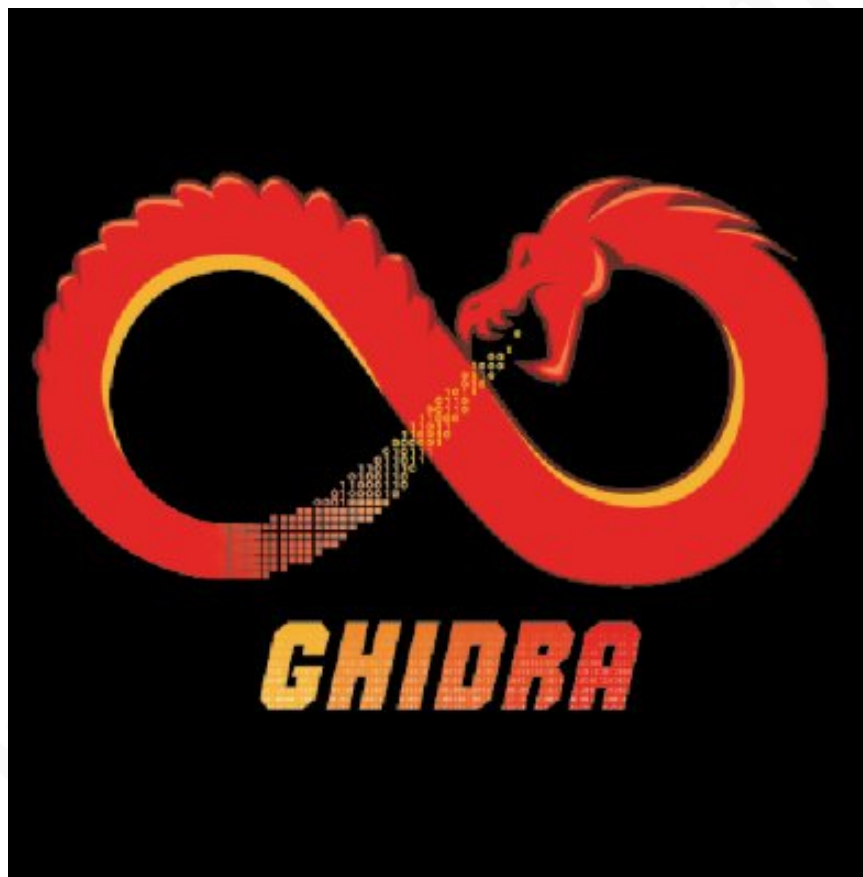
Aim

To perform static analysis of a suspicious binary file using Ghidra, understand its internal structure, and identify possible indicators of malicious behavior through disassembly and decompilation.

Procedure

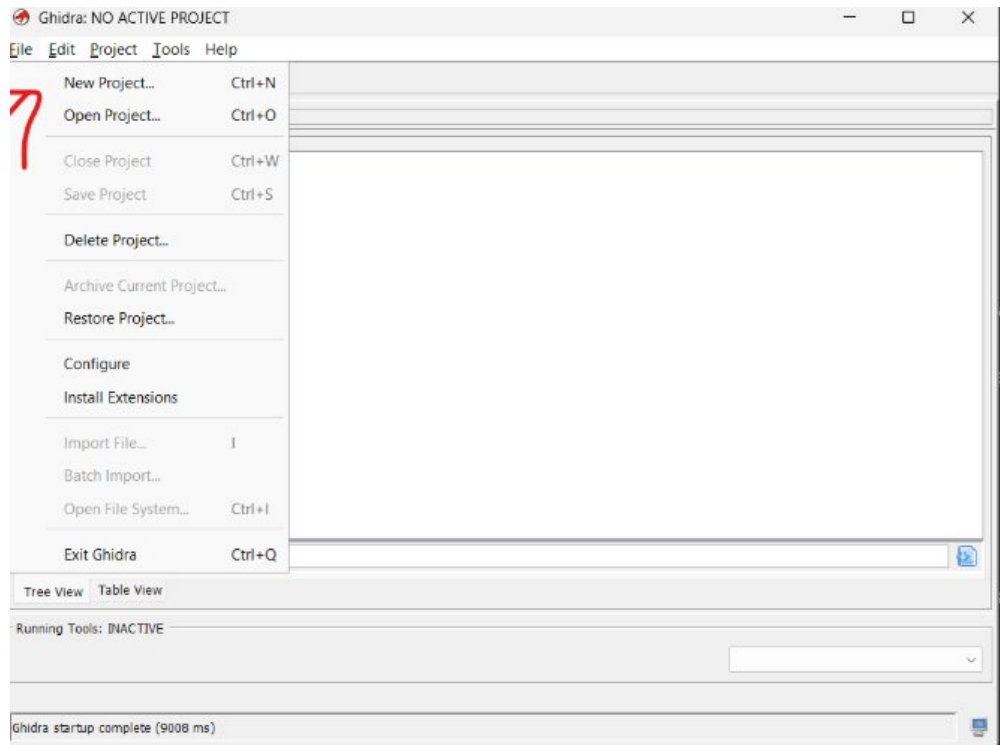
Step 1: Setting Up the Analysis Environment

- **Environment:** Launch your virtual machine to create an isolated, secure workspace.
- **Runtime:** Install and configure Ghidra, ensuring OpenJDK / JDK 17+ is properly linked.
- **Sample:** Prepare the binary sample to be analyzed.



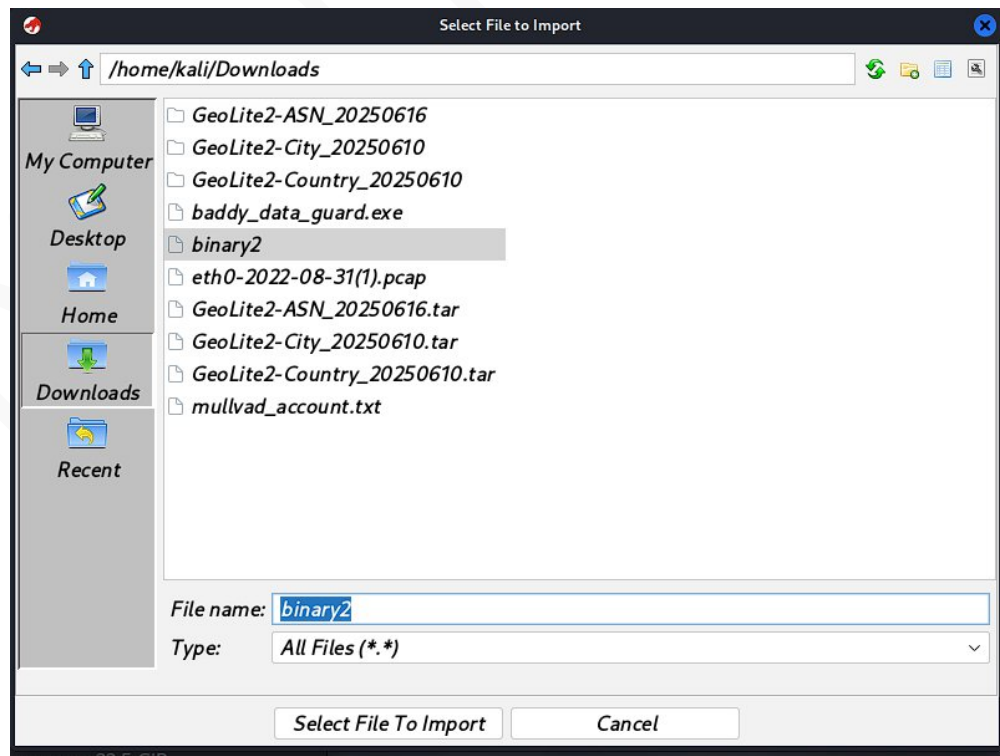
Step 2: Loading and Initial Analysis

- **Import:** Open Ghidra → Create a new project → Import the binary file.
- **Auto Analysis:** Run Auto Analysis and select all standard analyzers.
- **Entry Point:** Identify the entry point and inspect the disassembled machine code.



Step 3: Function Analysis & Decompilation

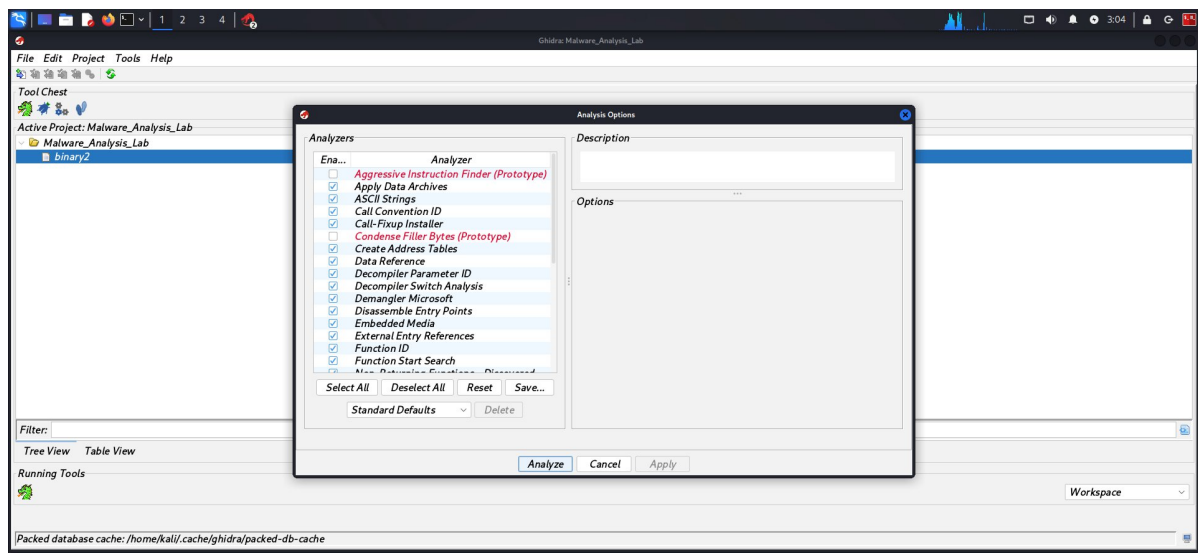
- **Decompilation:** Use the Decompiler to convert assembly into C-like code for improved readability.
- **Annotation:** Rename and document key functions to better understand program logic.
- **XREFs:** Use Cross-References (XREF) to track function calls and variable references.



Step 4: String and Import Table Analysis

Explore the Defined Strings section to locate readable strings such as C2 IP addresses, URLs, file paths, and registry entries.

Examine the Import Table for system API calls (e.g., CreateFileA, URLDownloadToFile, RegSetValueEx) which reveal malware capabilities.



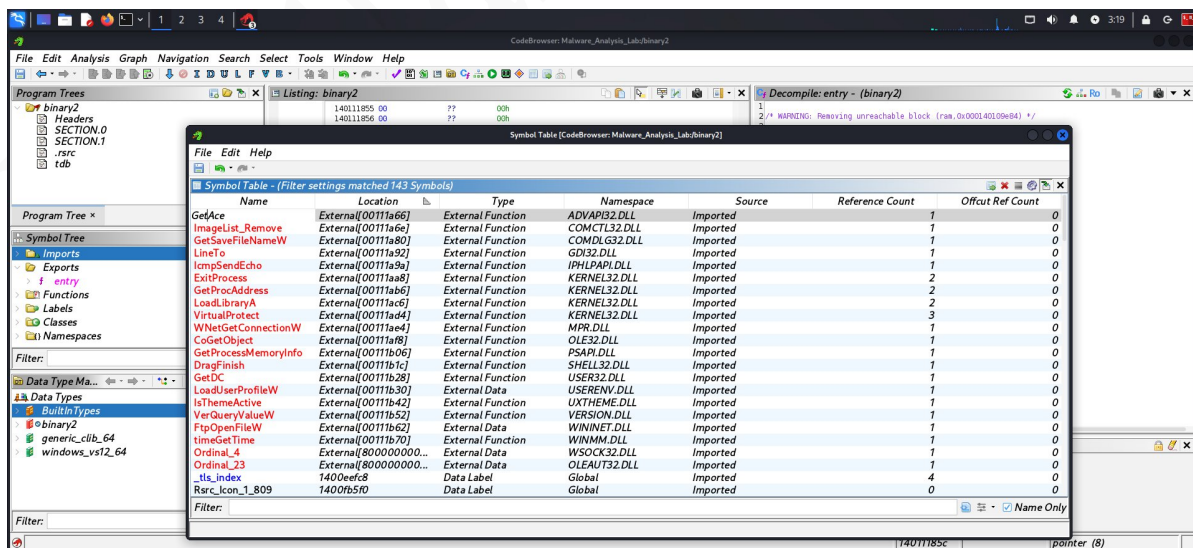
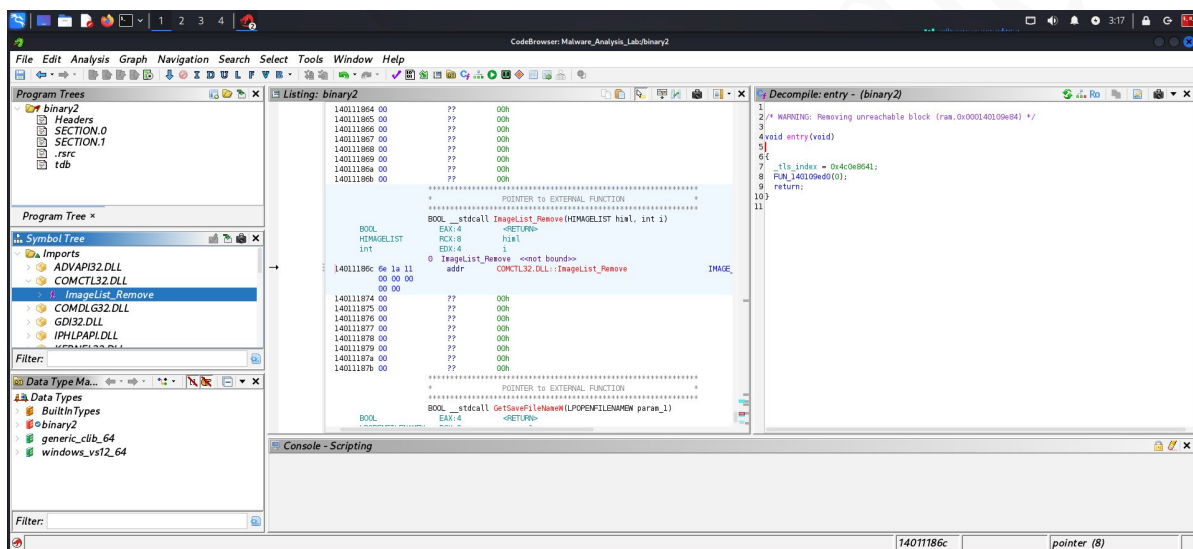
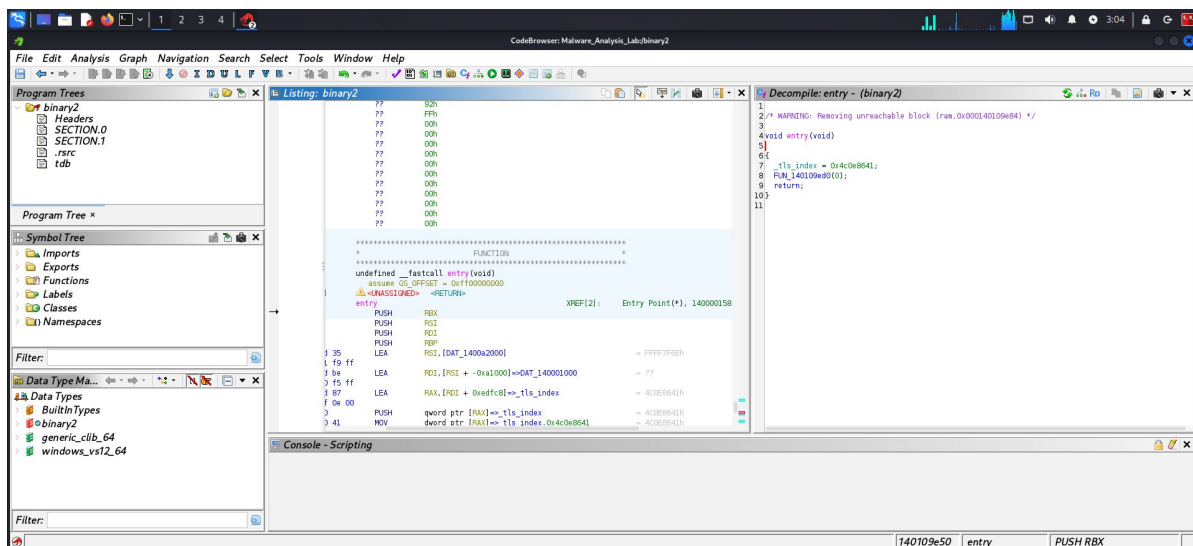
Step 5: Automation Scripts & Control Flow

Visualize the Control Flow Graph (CFG) to trace execution paths. Utilize automated Ghidra scripts for rapid triage:

Script Name	Forensic Description
extract_strings.py	Lists all unique string references found in the binary.
network_trace.py	Detects functions using networking APIs such as connect, send, or recv.
auto_label.py	Automatically labels known malicious patterns like registry changes or file manipulation.

Step 6: Demonstration Binary Disassembly

Reviewing disassembled functions and instruction traces from sample executable demonstration:



Step 7: Forensic Reporting & Indicator of Compromise (IOC) Format

- **1. Overview:** Description of the sample, architecture, and purpose of analysis.
- **2. Disassembly Observations:** List of key functions, control flow routines, and interpreted purpose.

- **3. Indicators of Compromise (IOCs):** Suspicious strings, network endpoints, or persistence hooks.
- **4. Conclusion & Recommendations:** Threat mitigation steps and remediation suggestions.

Rubrics

Criteria	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

Result

- The static analysis was successfully completed using Ghidra.
- The binary file was disassembled, examined, and its functions, decompiled C pseudocode, and strings were analyzed.
- Thus, the objective of performing malware analysis using Ghidra was achieved.